

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Кафедра цифрових технологій в енергетиці

Розрахункова-графічна робота

На тему: “Операції з текстурними координатами”

З дисципліни “Візуалізація графічної та геометричної інформації”

Варіант 20

Виконав:

Студент гр. ТР-51мп, НН ІАТЕ

Лудченко А.Є.

Перевірив:

Демчишин А. А.

Київ – 2025

Розділ 1. Опис завдання

1.1 Огляд

Метою цього завдання було розширення існуючого WebGL-додатку для підтримки розширених технік накладання текстур. Основне завдання полягає в реалізації можливості динамічного маніпулювання текстурними координатами (u , v) у реальному часі. Зокрема, додаток вимагав реалізації обертання текстури навколо певної, визначеної користувачем опорної точки, а також масштабування текстури.

1.2 Конкретні вимоги

Основними функціональними вимогами до проекту були:

1. Використати існуючу логіку накладання текстур для застосування 2D-зображення на 3D-параметричну поверхню (півсферу).
2. Реалізувати математичні перетворення для обертання та масштабування текстури.
3. На відміну від стандартного обертання, яке зазвичай відбувається навколо початку координат, ця реалізація вимагала обертання навколо довільної точки у просторі текстури.
4. Реалізувати обробку подій клавіатури, щоб дозволити користувачеві переміщати цю опорну точку по поверхні в реальному часі, вздовж осі U (горизонтально) та осі V (вертикально).

1.3 Мета

Кінцевим результатом є інтерактивна 3D-візуалізація, де користувач може інтуїтивно "пересувати" центр обертання по текстурованому 3D-об'єкту та спостерігати, як текстура огортається та обертається навколо цієї конкретної точки.

Розділ 2. Теоретичні відомості

2.1 Основи накладання текстур

Накладання текстур — це метод визначення високочастотних деталей, текстури поверхні або інформації про колір на комп'ютерній графіці або 3D-моделі. Він проектує 2D-зображення (текстуру) на 3D-поверхню.

Кожній вершині 3D-геометрії присвоюється текстурна координата, яку зазвичай позначають як u , v . Ці координати варіюються від 0.0 до 1.0.

- 0,0 представляє нижній лівий кут текстурного зображення.
- 1,1 представляє верхній правий кут.

Під час етапу растеризації графічний процесор інтерполює ці координати по поверхні трикутника, щоб отримати колір з текстурного зображення для кожного пікселя.

2.2 Трансформації текстурних координат

Так само, як 3D-геометрію можна трансформувати за допомогою матриць Model-View, текстурні координати можна трансформувати за допомогою матриць 3x3 або 2x2.

Обертання навколо початку координат

Стандартне 2D-обертання точки $P(u, v)$ на кут Θ навколо початку координат (0,0) визначається так:

$$\begin{aligned}u' &= u * \cos(\Theta) - v * \sin(\Theta) \\v' &= u * \sin(\Theta) + v * \cos(\Theta)\end{aligned}$$

Обертання навколо довільної точки

Складність цього завдання полягає в обертанні навколо конкретної центральної точки $C(c_x, c_y)$. Пряме застосування матриці обертання виконує поворот навколо початку координат (0,0). Щоб обертати навколо C , ми повинні виконати складену трансформацію:

1. Перемістити систему координат так, щоб точка C стала новим початком координат (0,0).

$$P_{temp} = P - C$$

2. Обертання (R): Обернути переміщену точку на кут Θ .

$$P_{rotated} = R * P_{temp}$$

3. Зворотна трансляція (T^{-1}): Перемістити систему координат назад у початкове положення.

$$P_{final} = P_{rotated} + C$$

Масштабування

Масштабування збільшує або зменшує повторення текстури. Подібно до обертання, масштабування концептуально повинно відбуватися відносно центральної точки, щоб запобігти "зміщенню" текстури від бажаного місця розташування.

$$P_{scaled} = P_{rotated} * ScaleFactor$$

2.3 Комбінований алгоритм

У Вершинному шейдері (Vertex Shader) кінцева операція для кожної текстурної координати вершини виглядає так:

```
vec2 centered = inTexCoords - texCenter; // Трансляція до початку координат
```

```
mat2 rotation = mat2(cos(a), -sin(a), sin(a), cos(a));
```

```
vec2 rotated = rotation * centered; // Обертання
```

```
vec2 scaled = rotated * texScale; // Масштабування
```

```
vec2 finalCoords = scaled + texCenter; // Зворотна трансляція
```

Розділ 3: Деталі реалізації

3.1 Огляд архітектури

Додаток створено з використанням чистого WebGL (JavaScript та GLSL). Архітектура розділена на три основні компоненти:

1. **main.js** - відповідає за життєвий цикл додатку, події введення та оркестрацію даних.
2. **shader.gpu** - містить код GLSL для Вершинного та Фрагментного шейдерів.
3. **model.js** - відповідає за генерацію геометрії параметричної поверхні.

3.2 Модифікація Вершинного шейдера (shader.gpu)

Стандартний вершинний шейдер було змінено для прийому трьох нових uniform змінних:

- uniform vec2 texCenter (координати (u,v) опорної точки).
- uniform float texAngle (кут обертання в радіанах).
- uniform float texScale (коефіцієнт масштабування).

Логіка, описана в розділі №2 "Теоретичні відомості", була реалізована безпосередньо у функції main() вершинного шейдера. Це гарантує, що трансформація текстури відбувається ефективно на GPU для кожної вершини перед растеризацією.

3.3 Логіка JavaScript (main.js)

Управління станом

Були введені глобальні змінні для відстеження стану трансформації:

```
let textureCenter = [0.5, 0.5];  
let textureAngle = 0.0;  
let textureScale = 1.0;
```

Обробка Uniform-змінних

Клас ShaderProgram та функція draw() були оновлені для передачі цих значень до GPU у кожному кадрі.

```
gl.uniform2fv(shProgram.iTexCenter, textureCenter);  
gl.uniform1f(shProgram.iTexAngle, textureAngle);  
gl.uniform1f(shProgram.iTexScale, textureScale);
```

Обробка введення

Додано прослуховувач подій `keydown` для перехоплення введення клавіш WASD. Крок зміни було встановлено на 0.05 для забезпечення плавного, але чутливого руху.

3.4 Генерація поверхні ([model.js](#))

Поверхня є параметричною півсферою. Текстурні координати генеруються лінійно на основі індексу циклу:

$$u = j / uSteps$$

$$v = i / vSteps$$

Це забезпечує стандартне UV-відображення, де текстура огортає півсферу один раз.

Розділ 4. Інструкція користувача

4.1 Огляд інтерфейсу

Після запуску `index.html` користувачеві надається 3D-вигляд текстурованої півсфери.

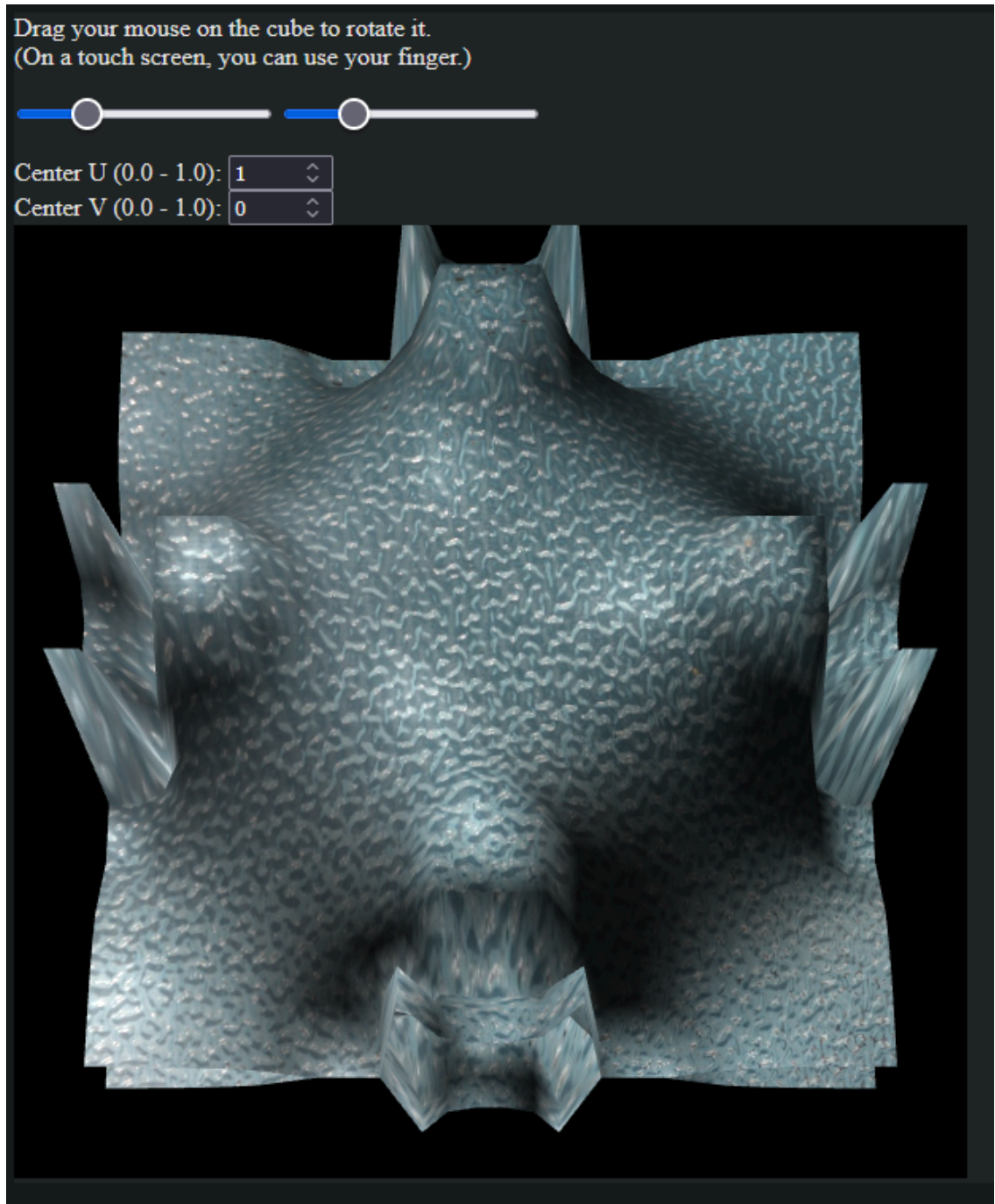


Рисунок 4.1 - Початковий вигляд додатку

4.2 Керування клавіатурою

Щоб маніпулювати точкою трансформації текстури, використовуйте клавіатуру. Переконайтеся, що вікно браузера активне, натиснувши один раз на полотні (canvas).

Клавіша	Дія	Візуальний ефект
W	Перемістити центр Вгору (+V)	Опорна точка візуально переміщується "вгору" по текстурі.
S	Перемістити центр Вниз (-V)	Опорна точка візуально переміщується "вниз" по текстурі.
A	Перемістити центр Вліво (-U)	Опорна точка візуально переміщується "вліво".
D	Перемістити центр Вправо (+U)	Опорна точка візуально переміщується "вправо".
Q	Обертати Проти годинникової стрілки	Обертає текстуру навколо поточного центру.
E	Обертати За годинниковою стрілкою	Обертає текстуру навколо поточного центру.

4.3 Приклад використання

1. Натисніть **Q**, щоб почати обертання текстури. Ви побачите, як вона обертається навколо центру зображення.
2. Натискайте **W** та **D** кілька разів.
3. Ви помітите, що центр "обертання" зміщується. Текстура тепер буде обертатися навколо нової точки (наприклад, верхнього правого кута), створюючи ефект "розгойдування", а не простого обертання.

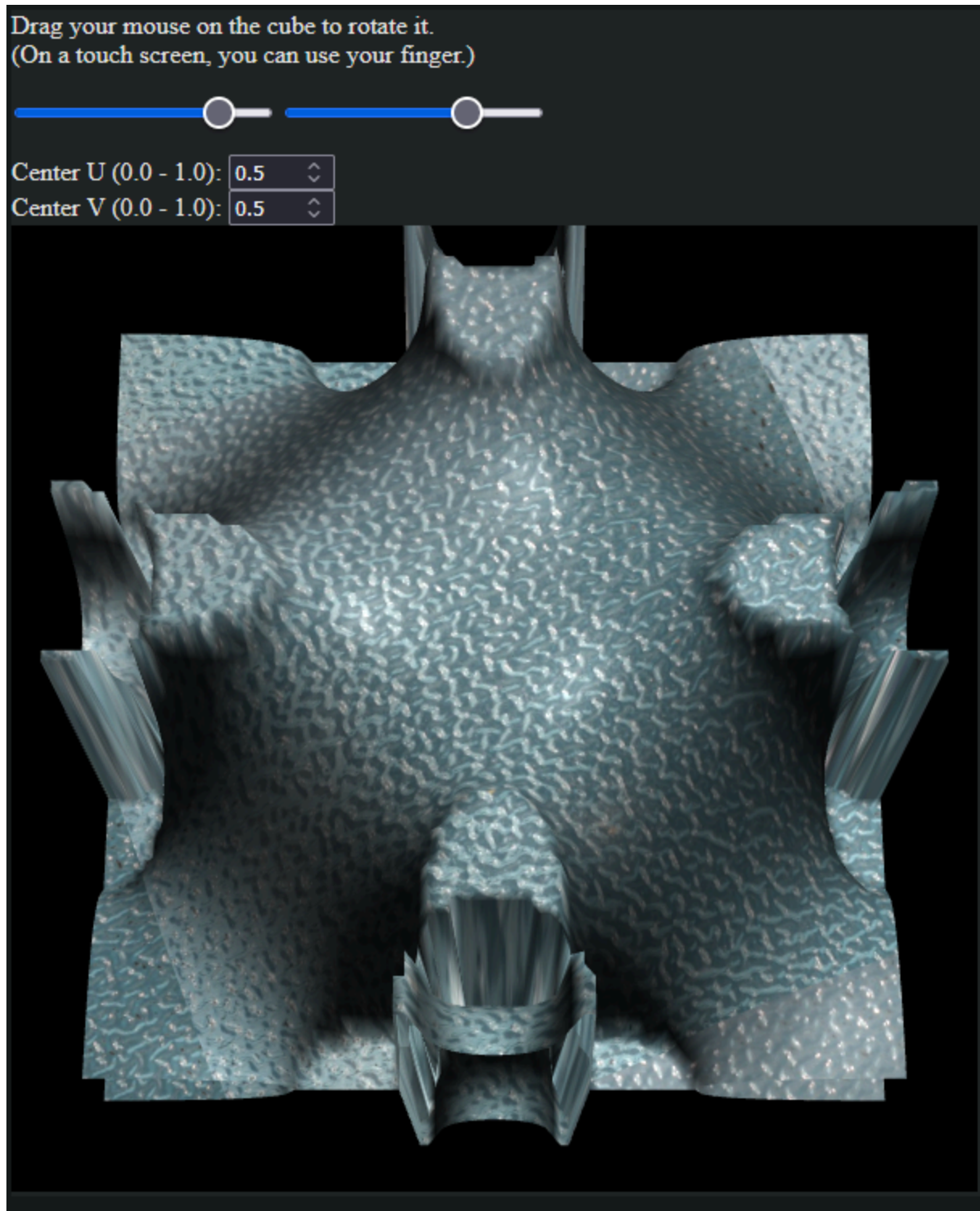


Рисунок 4.2 - Текстура, обернута навколо зміщеної точки

Розділ 5. Зразок вихідного коду

5.1 Вершинний шейдер (shader.gpu)

Цей розділ відповідає за основну математику трансформації.

```
attribute vec3 inVertex;  
attribute vec3 inNormal;  
attribute vec2 inTexCoords;  
  
// Вивід у фрагментний шейдер  
varying vec2 outTexCoords;  
  
// Uniform-змінні трансформації  
uniform mat4 ModelViewProjectionMatrix;  
uniform vec2 texCenter; // Центр, керований користувачем  
uniform float texAngle; // Кут обертання  
uniform float texScale; // Коефіцієнт масштабування  
  
void main() {  
    // 1. Зміщення центру до початку координат  
    vec2 centered = inTexCoords - texCenter;  
  
    // 2. Побудова матриці обертання  
    float s = sin(texAngle);  
    float c = cos(texAngle);  
    mat2 rotationMat = mat2(c, -s, s, c);  
  
    // 3. Застосування обертання та масштабування  
    vec2 rotated = rotationMat * centered;  
    vec2 scaled = rotated * texScale;  
  
    // 4. Зворотна трансляція до початкового центру  
    outTexCoords = scaled + texCenter;
```

```
// Стандартний розрахунок позиції вершини
gl_Position = ModelViewProjectionMatrix * vec4(inVertex, 1.0);
}
```

5.2 Обробка введення (main.js)

Ця функція JavaScript перехоплює введення користувача та оновлює центральні координати.

```
function handleKeyDown(event) {
    const step = 0.05;
    const inputU = document.getElementById("centerU");
    const inputV = document.getElementById("centerV");
    let changed = false;

    switch (event.key.toLowerCase()) {
        case "a":
            textureCenter[0] -= step;
            changed = true;
            break;
        case "d":
            textureCenter[0] += step;
            changed = true;
            break;
        case "s":
            textureCenter[1] -= step;
            changed = true;
            break;
        case "w":
            textureCenter[1] += step;
            changed = true;
            break;
        case "q":
            textureAngle += 0.1;
            draw();
            return;
        case "e":
```

```

        textureAngle -= 0.1;
        draw();
        return;
    }

    if (changed) {
        textureCenter[0] = Math.max(0, Math.min(1, textureCenter[0]));
        textureCenter[1] = Math.max(0, Math.min(1, textureCenter[1]));

        inputU.value = textureCenter[0].toFixed(2);
        inputV.value = textureCenter[1].toFixed(2);

        draw();
    }
}

window.addEventListener("keydown", handleKeyDown);

```

5.3 Цикл малювання (main.js)

Передача оновлених даних до GPU.

```

function draw() {
    gl.clearColor(0, 0, 0, 1);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
    //...
    gl.uniform2fv(shProgram.iTexCenter, textureCenter);
    gl.uniform1f(shProgram.iTexAngle, textureAngle);
    gl.uniform1f(shProgram.iTexScale, textureScale);
    //...
    surface.Draw();
}

```